PROJECT HANDOFF / 2026

A11yLens

Project Progress and Technical Handoff

Snapshot date: 30 October 2026
Latest GitHub baseline: `e9a6e50` on `main`
Local state: T1 popup wording and handoff documentation are not yet committed or pushed.
Project repository: https://github.com/lakshay74/a11ylens-accessibility-auditor
Project stage: Week 5 evaluation and interface polish; Week 6 release preparation started

CURRENT STAGE	Week 5 evaluation and interface polish; Week 6 release preparation started.
REPOSITORY	github.com/lakshay74/a11ylens-accessibility-auditor
RELEASE STATE	Local development project. No source license, hosted API, store release, or completed evaluation yet.

Inside this handoff

01 Why this report exists	02 Project summary
03 What is implemented	04 Privacy, security, and third-party handling
05 Development and repository structure	06 GitHub and project history
07 Roadmap status: completed, partial, and pending	08 Current limits and things not yet established
09 How to run the current local development version	10 Suggested handoff instructions for Claude
11 Repository documents to read next	12 Appendix A. WCAG reference corpus inventory
13 Appendix B. Curated axe rule mappings	

This report distinguishes repository implementation from features that still need user testing or release review.

1. Why this report exists

This is a handoff document for continuing A11yLens with another assistant or developer. It records what exists in the repository, how the parts fit together, decisions already made, what has been observed during setup, and what is still unfinished. The supplied six-week roadmap is tracked in `docs/ROADMAP.md`.

The user later supplied an independent review based on this report and four popup screenshots. That reviewer states that they did not inspect the repository or run the code. Screenshot observations are recorded as reported evidence; implementation comments were checked against the source before T1 changes. The review's task list is treated as recommendations, not as commands by itself.

Items in this report are classified as implemented, observed during setup, or still pending. A checked-off feature means source code or project material exists. It does not mean the feature has passed a formal test program, accessibility evaluation, security review, or store review.

2. Project summary

A11yLens is a Manifest V3 browser extension for inspecting accessibility issues on a page after the user explicitly starts a scan. The extension uses a locally bundled copy of axe-core. It displays automated violations separately from results that axe-core says need manual review, lets the user locate affected elements, and can request WCAG guidance about one selected finding from a local FastAPI service.

The FastAPI service works without an AI provider key. It retrieves local WCAG references and returns a short explanation. If the developer configures a Groq key in the local backend, the service can also request an AI-generated explanation and suggested HTML correction. Scanning does not depend on the backend being online.

The name is **A11yLens**: `a11y` is a common numeronym for accessibility, and `lens` describes looking closely at a page.

Item	Current state
Product	A11yLens - Accessibility Auditor
Browser package	Chrome/Brave Manifest V3 unpacked extension
Extension version	0.1.0
Git branch	`main`
Public source repository	`lakshay74/a11ylens-accessibility-auditor`
Latest commit on GitHub at the time of this report	`e9a6e50` - Link findings to axe rule guidance
Local changes after that baseline	T1 result labeling and handoff documentation; not committed or pushed
Backend	Python FastAPI, local-development service
AI provider	Optional Groq API; off unless `GROQ_API_KEY` is configured
WCAG reference set	55 WCAG 2.2 A/AA summaries linked to W3C WAI Understanding pages
Source license	Not selected
Store publication	Not done
Hosted API	Not deployed; local-only development setup

3. What is implemented

3.1 Browser extension foundation

- `extension/manifest.json` declares Manifest V3, the popup, action icons, and a background service worker.

- The extension has the permissions `activeTab`, `scripting`, and `storage`. The only host permission in the manifest is `http://127.0.0.1:8000/*` for the local explanation API.
- The popup is built from `popup.html`, `popup.css`, and `popup.js`. The current design uses a 390-pixel-wide, scrollable popup, a compact branded header, scan panel, status pill, result summary, finding cards, and action buttons.
- The popup shows whether the API is available. When the API is offline, the status says the scanner is ready and API is offline; the user can still scan because scanning runs in the browser.
- Summary tiles identify their units: violation rules, affected violation elements, and manual-review rules. The affected-element count covers violation results.
- Manual-review cards use a neutral **Possible issue** badge that includes axe's impact rating; only violation cards use severity colors. A manual-review card offers **Get WCAG guidance** with a clear note that it is help for reviewing a possible issue, not a confirmed failure. If axe returns the same rule in both groups, a note appears once for that rule.
- The page has status regions for scan and API updates, an alert region for errors, a visible keyboard focus style, and semantic headings and sections. These are implementation choices, not the outcome of a completed accessibility evaluation of the popup.
- The UI presents findings as text created with DOM APIs such as `textContent`; page-derived strings are not injected as popup HTML.
- Extension icons are checked in at 16, 32, 48, and 128 pixels. The Windows PowerShell source for regenerating them is `extension/scripts/build-icons.ps1`.

3.2 On-demand page scanning

1. The user opens the extension popup on an ordinary web page and clicks **Scan this page**.
2. `popup.js` sends `A11YLENS_SCAN_ACTIVE_TAB` to the service worker.
3. `background.js` checks the active tab and rejects unsupported browser pages, including Chrome internal pages, extension pages, the Chrome Web Store, and DevTools pages.
4. Only after the scan request, the worker injects the locally packaged axe-core bundle when needed, then injects `content.js`.
5. `content.js` runs axe-core against the page and returns results to the popup through the worker.
6. The service worker saves the scan under a per-tab key in `chrome.storage.session`, so the popup can restore the same page's scan if the popup closes and reopens during the browser session.

The axe run filters for `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, and `wcag22aa` tags and requests `violations` and `incomplete` result types. Violations appear as **rule violations**. Incomplete results appear as **manual review** and are not described as confirmed failures.

The three scan summary numbers have different units: `violations` counts violation rules, `affectedElements` counts nodes attached to violation results, and `incomplete` counts manual-review rules. `affectedElements` does not include nodes from incomplete results.

For each rule the content script keeps up to 25 affected nodes in the popup data. Each HTML sample is limited to 2,000 characters and each axe failure summary to 1,200 characters. The total affected-node count is retained separately, so the user can see the count even when only a sample of nodes is kept. The popup offers inspect buttons for up to three sample nodes per finding.

3.3 Finding inspection and help

- **Inspect element** sends the axe target selector to the content script. The content script scrolls the matching element into view, applies a visible outline, and stores the prior inline outline styles.
- **Clear outline** restores those inline outline and outline-offset values.
- The element lookup handles selector paths through open shadow roots where axe provides such a target. A target may no longer exist after page changes; the UI reports that condition. Some frame or shadow-DOM targets may not be highlightable.

- **View sample HTML** opens a details panel for the first captured node. This is a local display aid; the snippet is not included in JSON exports.
- Finding cards include an **Axe rule help** link only if the help URL uses HTTPS and its host is `dequeuniversity.com` or `www.dequeuniversity.com`. The link opens in a new tab with `noopener noreferrer`.
- A manual-review card's badge says **Possible issue** and shows axe's impact in neutral styling. A severity color is reserved for violation cards. Manual-review cards offer WCAG guidance with a disclosure that axe did not confirm a violation. If a rule appears in both result groups, one note explains that both results exist.

3.4 JSON scan export

The results panel includes **Export JSON**. The popup builds a versioned report with `format: "a11ylens-report-v1"`, a scan timestamp, page title, cleaned page URL, summary counts, and a list of findings. Each finding row contains category, axe rule ID, impact, description, affected-element count, and axe help URL.

Before export, the URL's username, password, query string, and fragment are removed. The JSON does not contain page HTML, selectors, or axe failure summaries. The browser creates a local download whose name includes the host and scan date, for example `a11ylens-example.com-2026-10-30.json`. A11yLens does not upload the exported file.

3.5 FastAPI guidance service

- `backend/main.py` creates the FastAPI application and defines `/health` and `POST /explain`.
- The documented development command binds Uvicorn to `127.0.0.1:8000`. No hosted A11yLens API exists in this project snapshot.
- The explanation request is a strict Pydantic model. Unexpected fields are rejected. Text fields have maximum lengths: rule ID 120 characters, description 500, impact 30, HTML 2,000, failure summary 1,200, and selector target 500.
- A request is sent only when a user presses **Get WCAG guidance** on a violation or manual-review card. A manual-review card labels the action as help for reviewing a possible issue and repeats that axe did not confirm a violation in the response. The extension sends the selected axe rule ID, description, impact, one short HTML snippet, one node's failure summary, and target selector to the local API. It does not send the result category. Page scanning itself sends nothing to the backend.
- With no `GROQ_API_KEY`, the service returns locally generated guidance based on retrieved WCAG records. The UI labels this as local guidance and does not pretend that an AI-generated code fix was returned.
- With a key, the API sends the selected finding and retrieved WCAG references from the backend to Groq. The prompt says page HTML is untrusted data, tells the model not to invent criterion numbers or make a conformance claim, and requests a JSON object.
- The backend only returns criterion IDs that were actually in the retrieved set and bounds generated string lengths before responding. It has a 35-second provider request timeout and maps timeout, provider rejection, network failure, and invalid JSON to API errors.
- The health route reports whether Groq is configured. The popup can show **AI explanations ready**, **Local explanations ready**, or **Scanner ready - API offline**.
- CORS accepts the extension origin pattern and local development origins (`localhost` and `127.0.0.1`).

3.6 WCAG retrieval and local reference corpus

- `backend/data/wcag.json` contains 55 records for WCAG 2.2 Level A and AA success criteria. Each record contains an ID, title, conformance level, concise plain-language summary, W3C WAI Understanding source URL, and a note that the source is informative guidance.
- The checked-in set currently contains 31 Level A records and 24 Level AA records. Appendix A lists the exact IDs and titles from the corpus at this snapshot.
- The corpus contains summaries from W3C WAI Understanding pages, not the complete normative WCAG standard. The project documentation says to follow the links and review the applicable criterion before making conformance decisions.

- `backend/scripts/build_wcag_corpus.py` can refresh the corpus. It reads the W3C index and pages, extracts the In Brief guidance, filters to A and AA criteria, retries page requests, sorts records, and writes a temporary file before replacing the corpus file.
- `backend/rag.py` has curated mappings for 16 common axe rule IDs. Known mappings are added first; the retrieval path then fills up to three criteria using semantic or lexical ranking.
- TF-IDF-style lexical retrieval is the default lightweight path and is available without the optional model packages.
- Optional local semantic retrieval uses Sentence Transformers with `sentence-transformers/all-MiniLM-L6-v2` and FAISS cosine-similarity search. The model is downloaded to the local model cache on first use, and the index is built in memory for the API process. If the packages or model fail to load, the code falls back to TF-IDF.
- `backend/requirements-rag.txt` holds the optional Sentence Transformers and FAISS dependencies separately from the basic backend requirements.

4. Privacy, security, and third-party handling

4.1 What stays local and what can leave the computer

- Starting a scan runs axe-core in the selected page through the extension. There is no A11yLens hosted service receiving the page.
- To restore results in the popup, `chrome.storage.session` temporarily holds scan data for the browser session. It can include page title and URL, findings, target selectors, HTML samples, and axe failure summaries. This is browser session storage, not a server-side A11yLens account or database.
- Export is generated locally. It removes URL credentials, query parameters, and fragments, and excludes HTML, selectors, and failure summaries.
- Requesting WCAG guidance sends the selected finding to the local FastAPI service. If a Groq key is configured, the selected finding and retrieved references are then sent by that service to Groq. Users should not send a finding if its content is not appropriate to share with the configured provider.
- The current build has no user accounts, analytics, or advertising.
- The draft disclosure is in `docs/PRIVACY.md`; it must be reviewed against the final code and any hosting changes before store publication.

4.2 Security and dependency notes

- The Groq secret belongs in `backend/.env`; `.gitignore` excludes `.env` and `backend/.env`. `backend/.env.example` is the configuration template.
- The extension has no API secret. The browser requests the local API only for an explicitly requested explanation.
- API input fields are strict and length-limited. Results from Groq are size-limited and their criterion IDs are checked against retrieved references.
- The backend development command is bound to loopback. It has not been hardened for public hosting; authentication, quotas, logging, deployment privacy, and provider costs need design review before any hosted deployment.
- `axe-core` is pinned as the npm build dependency (`4.14.0`). `extension/scripts/copy-axe.js` copies the package's scanner bundle and notices into `extension/vendor/` when building. The generated `axe.min.js` is ignored by Git; the license and third-party notices are checked in.
- The repository does not yet have an A11yLens project license. Public repository visibility alone does not grant an explicit reuse license. axe-core has its own separate license and notices.

5. Development and repository structure

Path	Purpose
`extension/manifest.json`	Manifest V3 metadata, permissions, popup, service worker, and icon references
`extension/popup.html`	Popup document structure and accessible status regions
`extension/popup.css`	Popup layout, typography, status styles, focus rings, finding cards, and severity labels
`extension/popup.js`	Popup messaging, rendering, local API calls, export, and local-file preview guard
`extension/background.js`	Active tab validation, on-demand script injection, message handling, and session scan storage
`extension/content.js`	axe-core scan, finding serialization, target highlighting, and outline cleanup
`extension/package.json` and `package-lock.json`	Pinned axe-core build dependency and build command
`extension/scripts/copy-axe.js`	Copies axe-core from npm into the generated extension bundle and copies notices
`extension/vendor/axe-core-LICENSE.txt`	axe-core license notice retained in source
`extension/vendor/axe-core-THIRD-PARTY-NOTICES.txt`	axe-core third-party notices retained in source
`extension/icons/`	Checked-in 16, 32, 48, and 128 pixel PNG icons
`extension/scripts/build-icons.ps1`	Windows script that regenerates the branded PNG icon set
`backend/main.py`	FastAPI routes, request validation, local responses, and optional Groq call
`backend/rag.py`	axe mappings, local TF-IDF ranking, and optional FAISS semantic retrieval
`backend/data/wcag.json`	Checked-in WCAG 2.2 A/AA W3C summary records
`backend/scripts/build_wcag_corpus.py`	W3C corpus refresh script
`backend/requirements.txt`	Basic API dependencies
`backend/requirements-rag.txt`	Optional embedding and FAISS dependencies
`backend/.env.example`	Safe template for local backend configuration
`README.md`	Overview, install/run instructions, privacy summary, status, and license note
`docs/ARCHITECTURE.md`	Beginner-oriented request flow, data shape, retrieval, export, and limits
`docs/ROADMAP.md`	Maintained six-week milestone checklist and release boundary
`docs/PROJECT_LOG.md`	Decisions, current implementation notes, local commands, and next work
`docs/PRIVACY.md`	Draft disclosure aligned to current local data flows
`docs/A11YLENS_PROJECT_PROGRESS_REPORT.md`	Detailed implementation, history, privacy, and roadmap handoff source
`output/pdf/A11yLens_Project_Progress_Report.pdf`	Shareable PDF version of this report
`CONTRIBUTING.md`	Build instructions, coding and privacy guidance, and issue process
`CHANGELOG.md`	User-visible changes recorded in an Unreleased section
`.github/ISSUE_TEMPLATE/bug_report.md`	Bug report template

Path	Purpose
`.github/ISSUE_TEMPLATE/feature_request.md`	Feature request template
`.gitignore`	Excludes virtual environments, secrets, node modules, and generated axe bundle

6. GitHub and project history

The public repository is `https://github.com/lakshay74/a11ylens-accessibility-auditor`. It was created at the user's request and confirmation. The local `main` branch tracks `origin/main`. The implementation history through the current snapshot contains these commits:

Commit	Date	Change
`748b32c`	2026-10-27	Initial A11yLens project foundation
`0cb25ca`	2026-10-27	Add A11yLens report export and release docs
`ab56162`	2026-10-27	Clarify local scanning and guidance status
`94d29cc`	2026-10-27	Clarify public repository and local backend status
`108a88c`	2026-10-28	Simplify README project notes
`e9a6e50`	2026-10-28	Link findings to axe rule guidance

The Git status was clean at the `e9a6e50` baseline before the handoff report and T1 wording updates. The handoff documents and T1 changes are now in the local working tree and have not been committed or pushed. `backend/.env` and API keys are excluded from commits. The public source repository does not mean that a license has been chosen, that the backend is public, or that the extension is published in a browser store.

Work completed during the project so far

- Established the product name, extension foundation, and a structured six-week roadmap.
- Implemented the on-demand scanner, result UI, inspection/highlighting controls, local explanation service, and WCAG reference retrieval.
- Added a versioned JSON export and documentation for architecture, privacy, contributions, changes, roadmap, and project decisions.
- Added branded extension icons, GitHub issue templates, and third-party axe-core license notices.
- Tracked the source in a public GitHub repository at the user's request.
- Refined the setup flow after the popup was first opened as a local file and displayed `Cannot read properties of undefined (reading 'sendMessage')`. The popup now detects when it is not running as an extension, disables the scan action, and explains how to load the unpacked extension in Brave.
- The user showed the extension listed in Brave's Extensions manager as A11yLens - Accessibility Auditor, version 0.1.0. That screenshot confirms extension loading, not scan behavior. The later review separately reports a scan on `claude.ai`; that scan was not reproduced in this session.
- Clarified the README so it describes the public source repository separately from the locally run API, and so it does not imply an unchosen source license.

7. Roadmap status: completed, partial, and pending

Week 1 - Extension foundation: implemented

Manifest V3, popup, background service worker, active-tab messaging, per-session scan storage, and a local-file preview/help state are present.

Week 2 - Accessibility scanning: implemented

axe-core is run on demand for WCAG 2.0, 2.1, and 2.2 A/AA tags. The UI separates violations and incomplete/manual-review results, shows counts and affected element samples, and supports highlighting.

Week 3 - Explanation API: implemented locally; follow-up work remains

The FastAPI API, request limits, no-key local WCAG guidance, and optional Groq explanation path are implemented. Still pending: a safe local API key walkthrough and clearer provider error/retry guidance in the user documentation.

Week 4 - WCAG retrieval: implemented; one setup decision remains

The local 55-record W3C summary corpus, common axe mappings, TF-IDF fallback, and optional local Sentence Transformers plus FAISS path are implemented. Still pending: deciding whether the heavier optional model packages should ever become part of the default developer setup.

Week 5 - Evaluation and interface polish: in progress

Completed: JSON report export, safe Deque axe-help links, GitHub maintenance docs, issue templates, API-offline/local-scan explanation, and T1 changes for manual-review labels and guidance, count units, and one-time duplicate-result notes.

Still pending:

- Verify local and AI guidance for `aria-required-children`, `color-contrast`, `target-size`, `label-content-name-mismatch`, and `aria-valid-attr-value`; record the returned criteria and errors. No such results are recorded yet.
- Create a small repeatable set of accessible and intentionally flawed demo pages.
- Record what the scanner catches and whether fixes improve those sample pages, while avoiding claims of automated WCAG conformance.
- Manually review popup keyboard use, screen-reader announcements, and layout with assistive technology.
- Refine report naming and presentation using evaluation feedback.

Week 6 - Release preparation: started

Completed: branded icon assets and a Windows regeneration script, plus a local-development privacy disclosure draft.

Still pending:

- Choose a source license deliberately and confirm the third-party notices for the intended distribution.
- Capture final extension screenshots in a clean browser profile on a demo page, with no personal data visible.
- Review permissions, privacy disclosures, key handling, and deployment risks against the release build.
- Decide whether the backend stays local-only or is redesigned as a secured hosted service.
- Prepare Chrome Web Store materials only if distribution is desired.

8. Current limits and things not yet established

- An automated axe scan is not a WCAG conformance verdict and cannot detect every accessibility problem.
- The corpus is a set of concise, informative W3C WAI Understanding summaries, not the full normative WCAG standard.
- Curated mappings cover common axe rules. Other rules rely on local retrieval and may need manual criterion review.
- AI explanations and suggested HTML use one selected node's limited context. A suggested correction needs a developer's review and may require more page context.
- Frame, shadow DOM, dynamically changing pages, and browser-protected pages can limit scanning or highlighting.
- No demo/repair evaluation set or measured coverage results are currently documented.

- The browser popup and extension have not completed a manual keyboard or screen-reader evaluation in this project record.
- The repo contains no automated test suite. The user reports testing the initial T1 labels on `https://penny-juice.com` successfully. The current popup JavaScript passes `node --check`, but the follow-up that shows the cross-category note once and adds manual-review guidance has not been exercised in Brave. The report PDF has been rendered and visually inspected.
- The attached review reports that four screenshots show a `claude.ai` scan with 3 violation rules, 4 affected violation elements, and 4 manual-review rules. This is a single observed page, not a repeatable evaluation, and this session did not reproduce it.
- The review warns that personal browser details, including an inbox count, private chat titles, and bookmarks, were visible in screenshots. Use a clean browser profile and a seeded demo page for future public captures.
- The review says inspect buttons, sample HTML, and axe-help links were visible, but their behavior was not shown. The export button was visible, but the downloaded report contents were not reviewed.
- The review observed the API-offline status, so it did not establish that local guidance or Groq explanations work end to end. It specifically recommends checking retrieval for the five axe rules listed under Week 5.
- `content.js` serializes axe rule IDs, descriptions, impacts, help URLs, nodes, and counts, but it does not include each rule's `tags` in the scan output. The raw axe result can provide criterion tags, but A11yLens would need to retain them before using them in an evaluation report.
- No source license, hosted backend, Chrome Web Store release, or formal privacy/store review has been completed.

9. How to run the current local development version

These commands are documented in the repository. They are included here to make the handoff self-contained; they have not been executed as part of preparing this report.

Build the extension scanner bundle

From the repository root in PowerShell:

```
Set-Location extension
npm install
npm run build
```

The build copies `axe.min.js` and its license notices from the installed npm dependency into `extension/vendor/`. The generated JavaScript bundle is ignored by Git and needs to be present locally before loading the unpacked extension.

Install and start the backend

From the repository root:

```
py -m venv backend/.venv
backend/.venv/Scripts/python.exe -m pip install -r backend/requirements.txt
Copy-Item backend/.env.example backend/.env
backend/.venv/Scripts/python.exe -m uvicorn main:app --app-dir backend --reload --host 127.0.0.1 --port
8000
```

Set `GROQ_API_KEY` in the local `backend/.env` only if AI explanations are wanted. The default model setting is `openai/gpt-oss-20b`; `GROQ_MODEL` can select another model supported by the Groq account. Without a key, local guidance remains available. Optional semantic retrieval can be installed using `backend/requirements-rag.txt`.

The local health URL is `http://127.0.0.1:8000/health`; FastAPI's interactive API documentation is `http://127.0.0.1:8000/docs`.

Load the extension in Brave or Chrome

1. Open `brave://extensions` in Brave or `chrome://extensions` in Chrome.

2. Enable Developer mode.
3. Choose **Load unpacked** and select this repository's `extension` directory.
4. Open an ordinary website, open the A11yLens toolbar popup, and click **Scan this page**.
5. Use **Inspect element**, **Axe rule help**, or **Get WCAG guidance** on a finding. The backend is needed only for the last action.

Opening `extension/popup.html` directly as a file is a static preview, not a running extension. Browser-internal pages, the Web Store, and some protected pages cannot be scanned. If Chrome reports that a content script is unavailable after reloading the extension, reload the website tab as well.

10. Suggested handoff instructions for Claude

Attach this report and share the public repository link. A useful first message is:

> This report is the current A11yLens implementation handoff. Please inspect the repository at https://github.com/lakshay74/a11ylens-accessibility-auditor and treat the six-week roadmap in `docs/ROADMAP.md` as the project's plan. Preserve the distinction between implemented, observed, and pending work. Do not claim accessibility conformance, a completed evaluation, a chosen license, a hosted API, or a store release unless the repository and evidence show that it has happened. Continue with the next unfinished roadmap work and keep the README, roadmap, project log, changelog, and this handoff aligned when project status changes. Explain technical work in beginner-friendly steps.

11. Repository documents to read next

For more context, read `README.md` for the overview and quick start; `docs/ROADMAP.md` for milestone status; `docs/ARCHITECTURE.md` for request flow; `docs/PROJECT_LOG.md` for decisions and next work; `docs/PRIVACY.md` for the data-flow draft; and `CONTRIBUTING.md` plus `CHANGELOG.md` for maintenance conventions and recorded changes.

Appendix A. WCAG reference corpus inventory

The following 55 IDs and titles are present in `backend/data/wcag.json` at the implementation snapshot. Every record includes its level, short W3C WAI Understanding summary, and source URL. The summaries are informative and do not replace the full WCAG success criteria.

Criterion	Level	Title
1.1.1	A	Non-text Content
1.2.1	A	Audio-only and Video-only (Prerecorded)
1.2.2	A	Captions (Prerecorded)
1.2.3	A	Audio Description or Media Alternative (Prerecorded)
1.2.4	AA	Captions (Live)
1.2.5	AA	Audio Description (Prerecorded)
1.3.1	A	Info and Relationships
1.3.2	A	Meaningful Sequence
1.3.3	A	Sensory Characteristics
1.3.4	AA	Orientation
1.3.5	AA	Identify Input Purpose
1.4.1	A	Use of Color
1.4.2	A	Audio Control

Criterion	Level	Title
1.4.3	AA	Contrast (Minimum)
1.4.4	AA	Resize Text
1.4.5	AA	Images of Text
1.4.10	AA	Reflow
1.4.11	AA	Non-text Contrast
1.4.12	AA	Text Spacing
1.4.13	AA	Content on Hover or Focus
2.1.1	A	Keyboard
2.1.2	A	No Keyboard Trap
2.1.4	A	Character Key Shortcuts
2.2.1	A	Timing Adjustable
2.2.2	A	Pause, Stop, Hide
2.3.1	A	Three Flashes or Below Threshold
2.4.1	A	Bypass Blocks
2.4.2	A	Page Titled
2.4.3	A	Focus Order
2.4.4	A	Link Purpose (In Context)
2.4.5	AA	Multiple Ways
2.4.6	AA	Headings and Labels
2.4.7	AA	Focus Visible
2.4.11	AA	Focus Not Obscured (Minimum)
2.5.1	A	Pointer Gestures
2.5.2	A	Pointer Cancellation
2.5.3	A	Label in Name
2.5.4	A	Motion Actuation
2.5.7	AA	Dragging Movements
2.5.8	AA	Target Size (Minimum)
3.1.1	A	Language of Page
3.1.2	AA	Language of Parts
3.2.1	A	On Focus
3.2.2	A	On Input
3.2.3	AA	Consistent Navigation
3.2.4	AA	Consistent Identification
3.2.6	A	Consistent Help
3.3.1	A	Error Identification

Criterion	Level	Title
3.3.2	A	Labels or Instructions
3.3.3	AA	Error Suggestion
3.3.4	AA	Error Prevention (Legal, Financial, Data)
3.3.7	A	Redundant Entry
3.3.8	AA	Accessible Authentication (Minimum)
4.1.2	A	Name, Role, Value
4.1.3	AA	Status Messages

Appendix B. Curated axe rule mappings

These are the direct mappings defined by `RULE_TO_CRITERIA` in `backend/rag.py`. Other axe findings are supplemented by TF-IDF or optional FAISS retrieval. A mapping is a starting reference, not an automatic conformance conclusion.

axe rule ID	Mapped WCAG criteria
`image-alt`	1.1.1
`image-redundant-alt`	1.1.1
`role-img-alt`	1.1.1
`color-contrast`	1.4.3
`document-title`	2.4.2
`link-name`	2.4.4
`heading-order`	1.3.1
`aria-required-children`	1.3.1
`html-has-lang`	3.1.1
`label`	3.3.2, 4.1.2
`select-name`	3.3.2, 4.1.2
`input-button-name`	4.1.2
`button-name`	4.1.2
`aria-input-field-name`	4.1.2
`aria-hidden-focus`	4.1.2
`duplicate-id-aria`	4.1.2

Handoff reminder: A11yLens is a working local development project, not a released or independently evaluated accessibility product. Update this report when implementation or release status changes.